LibMan

University Library Management System

Catalogue • Circulation • Digital Lending • Payments — one platform, fully deployed

Asante Gabriel Kwaku | Project Report Presentation | August 2026

Why LibMan?

Paper-based records
Manual borrowing logs are slow, and easy to lose or misfile.

No self-service
Every request needs a staff member at a desk to process.

No digital lending
No unified way to lend and read ebook titles online.

Manual fee collection
Fines and fees are tracked and collected by hand.

One platform

for physical circulation, digital lending, and payments — built and deployed end to end.

Project Objectives

Self-service borrowing
Browse, request, and manage loans without staff intervention.

Physical + digital
One platform for printed copies and ebook lending alike.

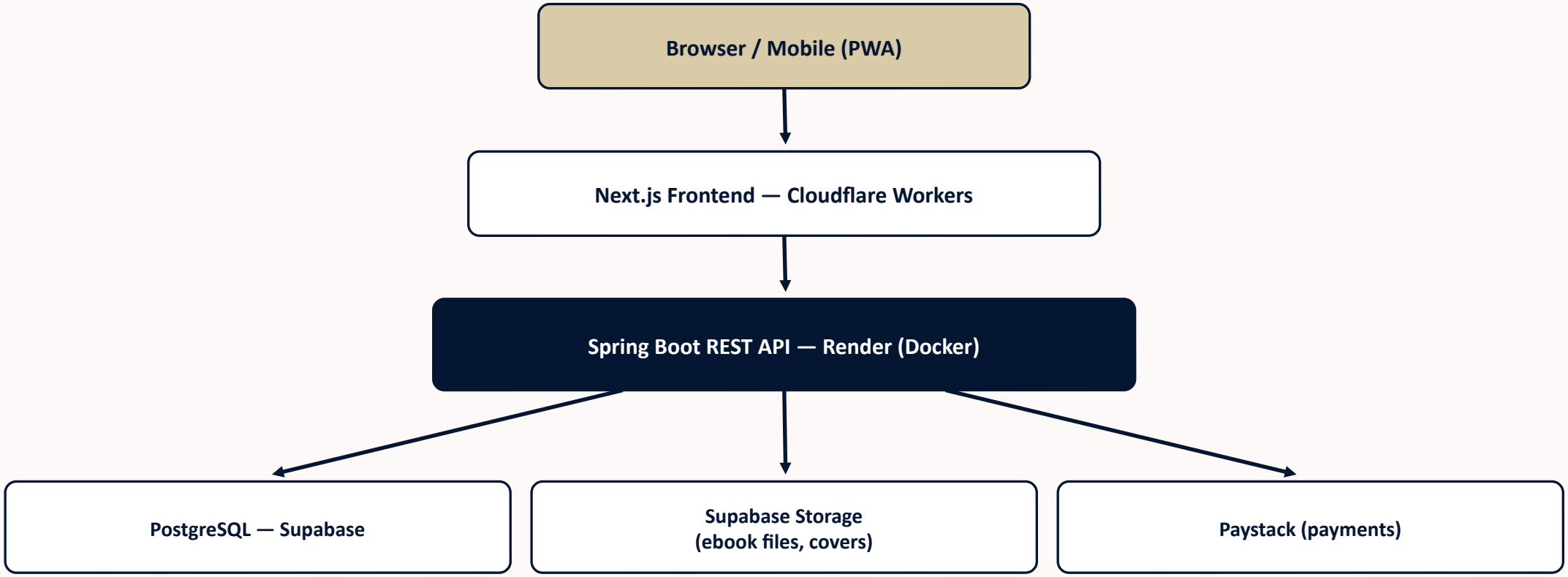
Automated fees
Real payment processing for fines and membership dues.

Librarian toolkit
Full catalogue, circulation, and user administration.

Consistent rules
Business rules enforced automatically, every time.

Always-on
Deployed to production, continuously reachable.

System Architecture



GitHub Actions + an external uptime monitor ping the backend's health endpoint every few minutes, so Render's free-tier inactivity spin-down never triggers.

Technology Stack — Frontend

Next.js 16
App Router, Turbopack

React 19 + TypeScript
Typed, component-driven UI

Tailwind CSS v4 + shadcn/ui
Design system on Base UI primitives

TanStack Query
Server-state caching & invalidation

React Hook Form + Zod
Typed, validated forms

Axios
Typed API client with auth interceptor

react-pdf
In-browser PDF rendering

epubjs
In-browser EPUB rendering

Technology Stack — Backend

Spring Boot 4.1
Layered controller / service / repository

Spring Security 7.1
JWT via OAuth2 Resource Server

Hibernate ORM 7.4
JPA persistence over PostgreSQL

PostgreSQL 17
Hosted on Supabase

Java 21 + Maven
Build and dependency management

Docker
Explicit, reproducible runtime image

Who Uses LibMan

Guest

- Browse the full catalogue
- Search titles & authors
- Register or sign in

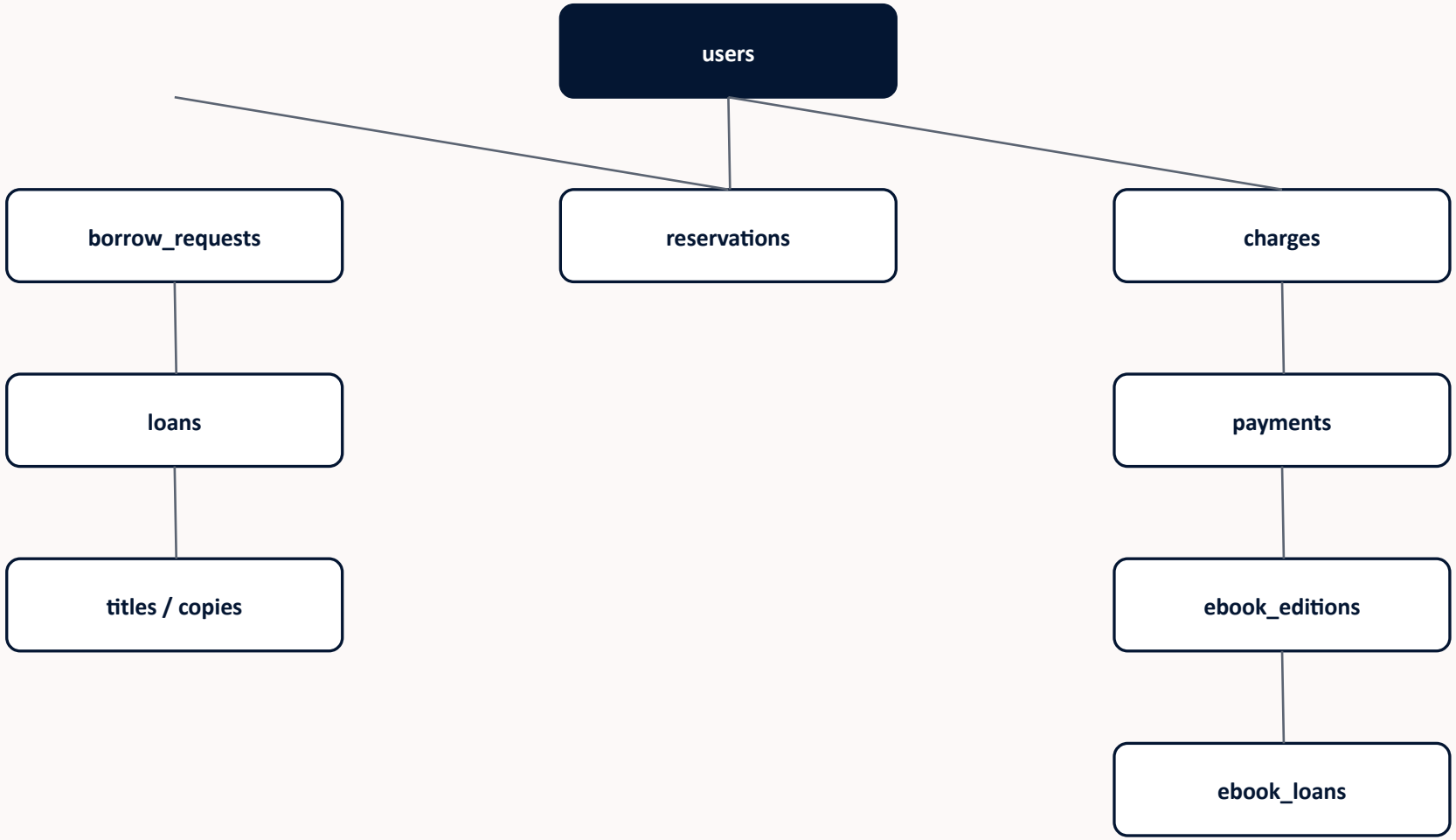
Student

- Borrow physical & ebook titles
- Choose loan duration (1 min–30 days)
- Reserve, extend, pay charges
- Read ebooks in-browser

Librarian

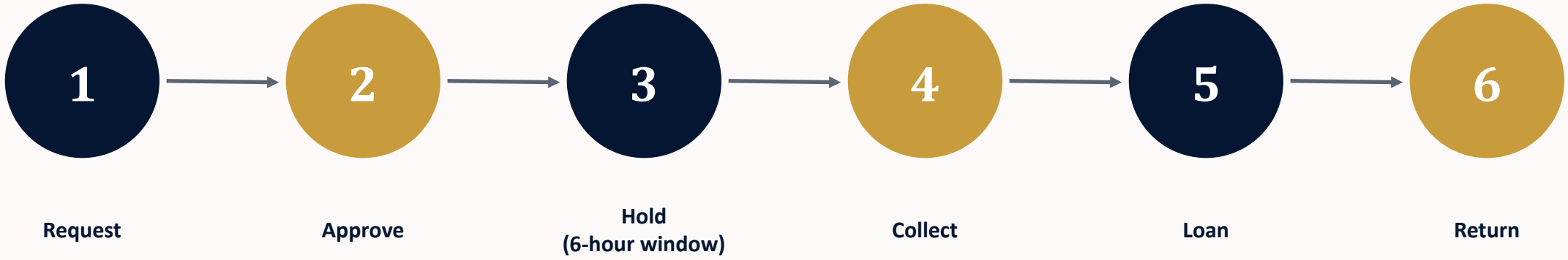
- Approve/reject requests
- Manage catalogue & covers
- Upload ebook files
- Process returns, users, reports

Database Design



Circulation state is driven largely by PostgreSQL triggers — approval creates the loan, collection sets the due date, and return releases the copy or advances the queue.

Borrowing Workflow



Approval opens a six-hour collection window on the held copy. If the item is never collected, a scheduled job automatically releases it — back to the catalogue, or to the next student in the reservation queue — at no cost to the student who never took it.

Borrower-Chosen Loan Duration

1 minute → 30 days

Students choose exactly how long they need an item at the moment they request it — for physical copies, or for ebooks. The due date is calculated automatically: for physical loans, from the moment the copy is actually collected, not from when the request was approved.

Digital Lending & In-Browser Reading

- Librarians upload a PDF or EPUB file per digital edition, stored securely in Supabase Storage.
- Students borrow with a self-chosen duration, exactly like physical items.
- Books render directly in the browser — no separate app or file download.
- Full-screen reading mode on both PDF and EPUB formats.

Charges & Fines

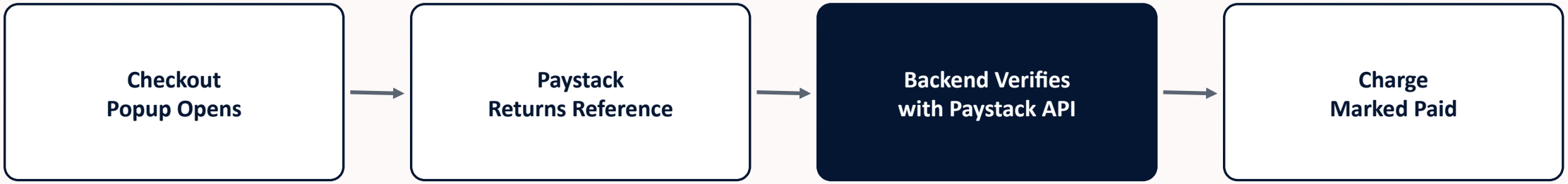
Late fee
Calculated from days overdue × a librarian-configurable per-day rate.

Damage / loss charge
The title's replacement cost, applied on a damaged or lost return.

Monthly membership fee
₵25, generated automatically for every student each month.

Unpaid = blocked
Any unpaid charge blocks all further borrowing — physical and digital — until cleared.

Secure Payments with Paystack

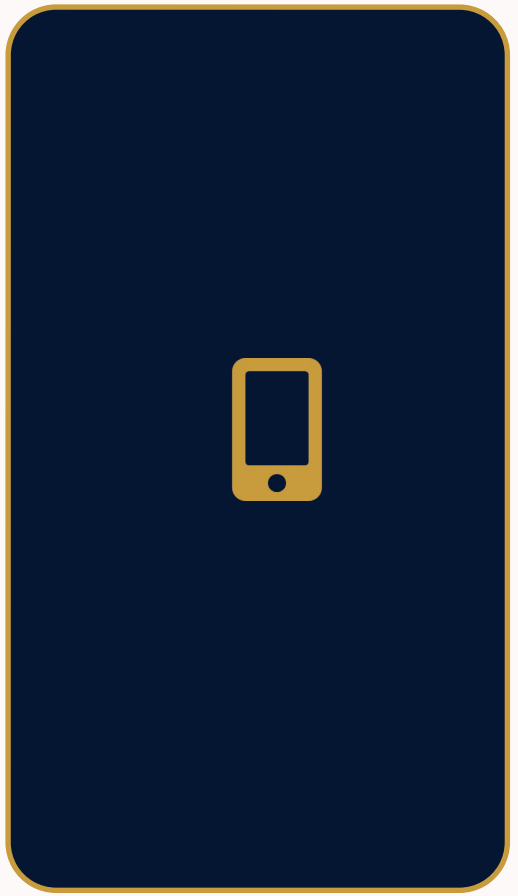


Payment success is never trusted from the client.

Every transaction reference is independently re-verified against Paystack's own API — checking both that it succeeded and that the amount paid exactly matches the charge — before anything is marked paid.

Installable on Any Device

- Add to Home Screen on Android and iOS alike.
- Own app icon, splash colours, and a real app name.
- Standalone display — opens without browser chrome.
- A service worker provides the offline shell installability requires.



Security by Design

Hashed passwords
Every password stored with bcrypt — never in plain text.

Stateless JWT auth
HS256-signed bearer tokens, scoped per request.

Secrets stay out of source control
Credentials and keys live only in environment configuration.

Server-verified payments
Payment success is confirmed against Paystack's API, never the client.

CORS allow-list
Only known, explicit frontend origins may call the API.

Testing Strategy

Unit testing
JUnit 5 + Mockito for pure business logic — late fees, replacement charges.

Static verification
TypeScript, ESLint, and a full production build on every change.

End-to-end verification
Real HTTP walkthroughs against live Supabase data and Paystack test mode.

Beyond the happy path
Deliberately reproduced real production bugs before fixing them.

Real Bugs, Real Fixes

Search endpoints returned HTTP 500

Postgres couldn't infer a bind parameter's type → explicit CAST in the query.

File uploads failed for every file

A manually-set header stripped the multipart boundary → let the browser set it.

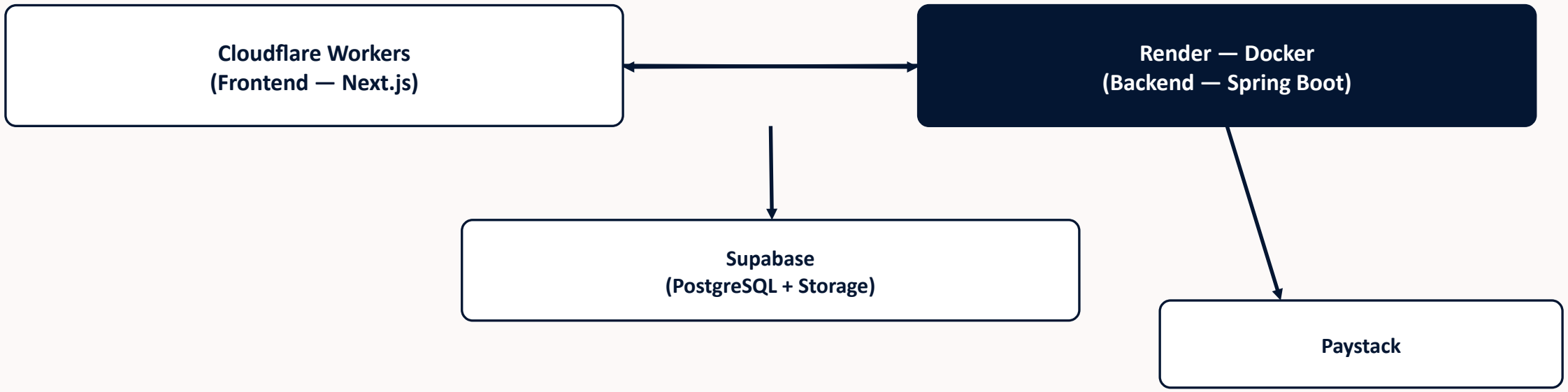
PDF reader showed huge blank gaps

One scanned page's size was reused for the whole book → each page now sizes independently.

Cloud build failed only on Linux

An optional dependency's lockfile entry was incomplete → pinned it explicitly.

Production Deployment



Continuous availability
GitHub Actions (every 10 minutes) and an external uptime monitor (every 5 minutes) both ping /actuator/health, so Render's 15-minute inactivity spin-down never fires.

What's Next

Current Limitations

- Reservation fulfilment doesn't yet offer a custom loan duration.
- Frontend deploys are triggered manually, not on every push.
- The service worker provides a basic offline shell only.

Future Enhancements

- Restore push-to-deploy once Cloudflare's build tooling stabilizes.
- Custom loan duration on reservation fulfilment.
- Expanded automated frontend test coverage.
- Push notifications for due-date reminders.

Built. Tested. Deployed.

LibMan delivers a complete library management system — catalogue, circulation, digital lending, real payments, and librarian administration — running in production on cloud infrastructure today.

Thank you.

github.com/AsanteAigu/LibMan